Chapter 16 — Cluster Management and Multi-Tenant Systems

> *"A 10,000-GPU cluster running at 60% utilization costs the same as one running at 85% utilization. The difference is the scheduler. Getting scheduling right is worth more than the next GPU generation."* — Production AI Infrastructure Maxim

---

Chapter Overview

A Principal Performance Architect who can design the perfect GPU kernel but cannot schedule, monitor, and operate a GPU cluster at scale is only half-equipped. Cluster management determines how efficiently hardware investments are realized: idle GPUs, mis-scheduled jobs, undetected node failures, and poor preemption policies all translate directly into wasted capital.

This chapter provides the complete production guide for the two dominant cluster management systems in AI infrastructure: **SLURM** (the standard for HPC and large-scale training) and **Kubernetes** (the dominant platform for inference and cloud-native AI).

Sections in this chapter:

- 16.1 SLURM Architecture and the Three Core Daemons
- 16.2 Production SLURM Job Scripts — Multi-Node GPU Training
- 16.3 SLURM Advanced — QOS, Fair-Share, and Priority Tuning
- 16.4 Topology-Aware Scheduling and Gang Scheduling
- 16.5 Kubernetes for AI — Architecture and GPU Resource Model
- 16.6 Kubernetes Production Patterns — Inference and Training
- 16.7 Resource Isolation and Multi-Tenancy
- 16.8 Fault-Tolerant Training — Checkpoint-Restart Automation
- 16.9 SLURM vs Kubernetes — Decision Framework
- 16.10 Production Operations Playbook
- 16.11 Key Takeaways and Review Questions

16.1 SLURM Architecture and the Three Core Daemons

16.1.1 The SLURM Daemon Hierarchy

```
SLURM DAEMON ARCHITECTURE:

┌──────────────────────────────────────────────────────────────────────┐
│  HEAD NODE (Control Plane)                                           │
│                                                                      │
│  ┌──────────────────────────┐   ┌──────────────────────────────┐    │
│  │  slurmctld               │   │  slurmdbd                    │    │
│  │  (Controller Daemon)     │   │  (Database Daemon)           │    │
│  │                          │   │                              │    │
│  │  • Job queue management  │   │  • Accounting records        │    │
│  │  • Resource tracking     │   │  • Usage history             │    │
│  │  • Scheduling decisions  │   │  • Fair-share computation    │    │
│  │  • Node state management │   │  • Job completion records    │    │
│  │  • Priority calculation  │   │  • MySQL/MariaDB backend     │    │
│  └──────────┬───────────────┘   └──────────────────────────────┘    │
└─────────────┼────────────────────────────────────────────────────────┘
              │ TCP/IP RPC communication
              │
    ┌─────────┴──────────────────────────────────────────┐
    │              COMPUTE NODES                         │
    │  ┌──────────────┐   ┌──────────────┐   ...        │
    │  │  slurmd      │   │  slurmd      │              │
    │  │  (Node Agent)│   │  (Node Agent)│              │
    │  │              │   │              │              │
    │  │ • Execute    │   │ • Execute    │              │
    │  │   job steps  │   │   job steps  │              │
    │  │ • Report     │   │ • Report     │              │
    │  │   resources  │   │   resources  │              │
    │  │ • Health     │   │ • Health     │              │
    │  │   monitoring │   │   monitoring │              │
    │  └──────────────┘   └──────────────┘              │
    └────────────────────────────────────────────────────┘

KEY CONCEPTS:
  Cluster:    All nodes managed by one slurmctld
  Partition:  Named group of nodes sharing a scheduling policy (like a queue)
              Examples: "gpu_training", "gpu_inference", "debug"
  Job:        Resource allocation request (nodes, GPUs, walltime)
  Job Step:   Actual execution unit within a job (srun command)
```

```
  GRES:       Generic Resources — how GPUs are specified and tracked
              Example: --gres=gpu:h100:8 (8 H100 GPUs per node)
```

16.1.2 SLURM Configuration for GPU Clusters

```bash
# /etc/slurm/slurm.conf — essential GPU-cluster settings

# Scheduling policy
SchedulerType=sched/backfill        # Backfill: fills gaps with smaller jobs
SelectType=select/cons_tres         # Track CPUs, RAM, GPUs per-job
SelectTypeParameters=CR_Core        # Allocate at CPU core granularity

# GPU GRES configuration
GresTypes=gpu                        # Enable GPU resource tracking
# Each node entry:
# NodeName=gpu001 Gres=gpu:h100:8 CPUs=112 RealMemory=2000000 State=IDLE

# Priority configuration
PriorityType=priority/multifactor   # Multi-factor priority (fair-share + age + QOS)
PriorityDecayHalfLife=7-00:00:00   # Usage halves every 7 days
PriorityWeightFairshare=10000       # Fair-share is the dominant factor
PriorityWeightAge=1000              # Waiting time bonus
PriorityWeightQOS=5000              # QOS tier matters

# /etc/slurm/gres.conf — per-node GPU configuration (on every compute node)
# AutoDetect=nvml    # Let SLURM auto-detect NVIDIA GPUs via NVML
# If manual: Name=gpu Type=h100 File=/dev/nvidia[0-7]

# Verify GPU detection:
# scontrol show node gpu001 | grep Gres
# Gres=gpu:h100:8
```

16.2 Production SLURM Job Scripts

16.2.1 Multi-Node GPU Training — Complete Script

```bash
```

```bash
#!/bin/bash
#═══════════════════════════════════════════════════════════════════════
# PRODUCTION SLURM JOB SCRIPT: Multi-Node LLaMA Training
# Usage: sbatch train_llama70b.sh
#═══════════════════════════════════════════════════════════════════════

# ── RESOURCE ALLOCATION ──────────────────────────────────────────────
#SBATCH --job-name=llama3-70b-train
#SBATCH --partition=gpu_training         # Use training partition
#SBATCH --nodes=64                       # 64 nodes × 8 GPUs = 512 GPUs
#SBATCH --ntasks-per-node=8             # One task (rank) per GPU
#SBATCH --cpus-per-task=14              # 14 CPU cores per GPU task
#SBATCH --gres=gpu:h100:8               # Request 8 H100s per node
#SBATCH --mem=0                          # Use all available memory per node

# ── TIME AND SCHEDULING ──────────────────────────────────────────────
#SBATCH --time=72:00:00                  # 72-hour walltime limit
#SBATCH --qos=training_standard          # QOS tier
#SBATCH --account=ml_team_budget         # Charge to this account

# ── FAULT TOLERANCE ──────────────────────────────────────────────────
#SBATCH --requeue                         # Automatically requeue on preemption
#SBATCH --signal=USR1@120               # Send SIGUSR1 120s before walltime
                                          # Training script saves checkpoint on SIGU

# ── TOPOLOGY ─────────────────────────────────────────────────────────
#SBATCH --switches=8@00:30:00           # Request ≤8 IB switches; wait ≤30 min
                                          # Ensures max 2-hop communication

# ── OUTPUT ───────────────────────────────────────────────────────────
#SBATCH --output=/lustre/logs/%j/train_%j.log
#SBATCH --error=/lustre/logs/%j/train_%j.err
#SBATCH --mail-type=FAIL,END             # Email on failure or completion

#═══════════════════════════════════════════════════════════════════════
# ENVIRONMENT SETUP
#═══════════════════════════════════════════════════════════════════════
set -euo pipefail                        # Fail fast on any error

module load cuda/12.3 nccl/2.20 openmpi/4.1

# ── CRITICAL NCCL CONFIGURATION (Chapter 14) ─────────────────────────
export NCCL_IB_HCA=mlx5_0:1,mlx5_1:1,mlx5_2:1,mlx5_3:1,mlx5_4:1,mlx5_5:1,mlx5_6:1,ml
```

```bash
export NCCL_IB_QPS_PER_CONNECTION=4
export NCCL_NET_GDR_LEVEL=SYS
export NCCL_NVLS_ENABLE=1
export NCCL_SHARP_ENABLE=1
export NCCL_TIMEOUT=1800                  # 30 min NCCL watchdog (large clusters)
export NCCL_DEBUG=WARN                    # Production: WARN (not INFO)

# ── GPU PERFORMANCE SETTINGS ──────────────────────────────────────
export CUDA_DEVICE_MAX_CONNECTIONS=1      # Required for some NCCL patterns
export CUDA_LAUNCH_BLOCKING=0             # Async GPU launches (default, confirm)

# ── PYTORCH SETTINGS ──────────────────────────────────────────────
export PYTHONFAULTHANDLER=1               # Python stack trace on segfault
export OMP_NUM_THREADS=14                # Match --cpus-per-task

# ── JOB INFORMATION ───────────────────────────────────────────────
MASTER_NODE=$(scontrol show job $SLURM_JOB_ID | grep " NodeList" | \
              awk '{print $1}' | sed 's/NodeList=//')
MASTER_ADDR=$(srun --nodes=1 --ntasks=1 -w $MASTER_NODE hostname --ip-address)
MASTER_PORT=29500
NNODES=$SLURM_NNODES
NPROC_PER_NODE=8
WORLD_SIZE=$(( NNODES * NPROC_PER_NODE ))

echo "Job $SLURM_JOB_ID: $NNODES nodes × $NPROC_PER_NODE GPUs = $WORLD_SIZE ranks"
echo "Master: $MASTER_ADDR:$MASTER_PORT"

#═══════════════════════════════════════════════════════════════════
# LAUNCH DISTRIBUTED TRAINING
#═══════════════════════════════════════════════════════════════════
srun --kill-on-bad-exit=1 \              # Kill all ranks if any rank fails
     --export=ALL \                       # Pass all env vars to tasks
     torchrun \
       --nproc_per_node=$NPROC_PER_NODE \
       --nnodes=$NNODES \
       --node_rank=$SLURM_NODEID \
       --master_addr=$MASTER_ADDR \
       --master_port=$MASTER_PORT \
       --rdzv_backend=c10d \
       train.py \
         --model llama3-70b \
         --data /lustre/datasets/pretraining/ \
         --checkpoint_dir /lustre/checkpoints/$SLURM_JOB_ID/ \
         --resume_from_latest true
```

```bash
#==============================================================================
# POST-JOB CLEANUP (runs after srun exits)
#==============================================================================
echo "Job $SLURM_JOB_ID completed at $(date)"
sacct -j $SLURM_JOB_ID --format=JobID,Elapsed,MaxRSS,ExitCode
```

16.2.2 Prologue and Epilogue Scripts

```bash
```

```bash
#!/bin/bash
# /etc/slurm/prolog.sh — Runs on compute node BEFORE each job
# Validates GPU health and configures performance state

set -euo pipefail

# ── 1. Validate GPU Health ──────────────────────────────────────
echo "PROLOG: Checking GPU health for job $SLURM_JOB_ID on $(hostname)"

GPU_COUNT=$(nvidia-smi -L | wc -l)
if [ "$GPU_COUNT" -ne 8 ]; then
    echo "ERROR: Expected 8 GPUs, found $GPU_COUNT"
    exit 1  # SLURM marks node as failed, reschedules job
fi

# Check ECC errors (uncorrectable = bad GPU)
UNCORR_ECC=$(nvidia-smi --query-gpu=ecc.errors.uncorrected.volatile.total \
             --format=csv,noheader,nounits | awk '{sum+=$1} END {print sum}')
if [ "$UNCORR_ECC" -gt 0 ]; then
    echo "ERROR: Uncorrectable ECC errors: $UNCORR_ECC — draining node"
    scontrol update NodeName=$(hostname) State=DRAIN \
        Reason="Uncorrectable ECC errors detected by prolog"
    exit 1
fi

# ── 2. Set Performance State ────────────────────────────────────
sudo nvidia-smi --persistence-mode=1    # GPU stays in P0 between requests
sudo nvidia-smi -pl 700                 # Set 700W power limit (full TDP)
sudo nvidia-smi -lgc 1785,1980          # Allow boost clocking (1785-1980 MHz)

# ── 3. Exclusive Process Mode ───────────────────────────────────
sudo nvidia-smi -c EXCLUSIVE_PROCESS   # One process per GPU (training only)

echo "PROLOG: GPU health OK, performance configured"
exit 0

#──────────────────────────────────────────────────────────────────
#!/bin/bash
# /etc/slurm/epilog.sh — Runs AFTER each job, resets GPU state

# Reset compute mode for next job
sudo nvidia-smi -c DEFAULT             # Allow multiple processes again
sudo nvidia-smi --persistence-mode=0  # Re-enable power saving between jobs
```

```bash
# Collect GPU utilization accounting
DCGM_JOB_STATS=$(dcgmi job --stop $SLURM_JOB_ID 2>/dev/null | \
                 grep "Avg.*GPU" || echo "DCGM not available")
echo "Job $SLURM_JOB_ID GPU stats: $DCGM_JOB_STATS"

# Kill any orphaned processes
nvidia-smi | grep -E "^\| +[0-9]+ +[0-9]+ +[0-9]+" | \
    awk '{print $5}' | xargs -r kill -9 2>/dev/null || true

exit 0
```

16.3 SLURM Advanced — QOS, Fair-Share, and Priority Tuning

16.3.1 Quality of Service Configuration

```bash
```

```bash
# ── MULTI-TIER QOS DESIGN ──────────────────────────────────────────

# Tier 1: Debug/interactive (fast queue, tiny limits)
sacctmgr add qos name=debug \
    Priority=100 \
    MaxTRESPU="cpu=112,gres/gpu=8" \     # Max 8 GPUs per user at once
    MaxJobsPerUser=5 \
    MaxWallDurationPerJob=02:00:00 \     # 2 hours max
    GraceTime=120                         # 2-min grace before kill on timeout

# Tier 2: Standard research
sacctmgr add qos name=research \
    Priority=50 \
    MaxTRESPU="cpu=896,gres/gpu=64" \
    MaxJobsPerUser=20 \
    MaxWallDurationPerJob=7-00:00:00     # 7 days max

# Tier 3: High-priority production (can preempt research)
sacctmgr add qos name=production \
    Priority=200 \
    MaxTRESPU="cpu=3584,gres/gpu=256" \
    Preempt=research,debug \             # Can preempt lower tiers
    PreemptMode=requeue \                # Requeue (not cancel) preempted jobs
    MaxWallDurationPerJob=30-00:00:00

# Create account hierarchy
sacctmgr add account name=ml_team \
    Description="ML Research Team" Organization="Engineering"

sacctmgr add user venkat Account=ml_team \
    DefaultQOS=research \
    QOS="debug,research,production"

# Run a job at specific QOS:
# sbatch --qos=production --account=ml_team train_large.sh
```

16.3.2 Fair-Share Scheduling

```bash
```

```bash
# VIEW FAIR-SHARE STANDINGS:
sshare -a -l --format=Account,User,RawShares,NormShares,RawUsage,EffectvUsage,FairSh

# INTERPRETATION:
# FairShare → 1.0: User has used much LESS than their share → HIGH priority
# FairShare → 0.0: User has used much MORE than their share → LOW priority
# This prevents any one team from monopolizing the cluster long-term.

# Example output:
# Account    User     RawShares NormShares RawUsage  FairShare
# ml_team             500       0.5        12500000  0.55
#   venkat           100       0.2        8000000   0.45   ← heavy user, lower prior
#   alice            100       0.2        4000000   0.62   ← light user, higher prio

# KEY slurm.conf FAIR-SHARE PARAMETERS:
# PriorityDecayHalfLife=7-00:00:00    # Usage halves every 7 days (rolling memory)
# PriorityWeightFairshare=10000        # Fair-share dominates over age/QOS
# PriorityWeightAge=1000               # Waiting time adds small boost
# PriorityWeightQOS=5000               # QOS tier has meaningful weight

# CLUSTER EFFICIENCY REPORTING:
# Jobs with GPU allocation in last 7 days:
sacct --starttime=now-7days \
      --format=User,JobID,AllocTRES,Elapsed,State \
      --allusers | grep "gres/gpu"

# Total GPU-hours consumed per user:
sacct --starttime=now-30days --allusers \
      --format=User,AllocTRES,Elapsed --parsable2 | \
      awk -F'|' '{match($2, /gres\/gpu=([0-9]+)/, g); \
      split($3, t, ":"); h=t[1]*1+t[2]/60; print $1, g[1]*h}' | \
      sort -k2 -nr | head -20
```

16.4 Topology-Aware Scheduling and Gang Scheduling

16.4.1 Topology-Aware Placement

```bash
```

```bash
# TOPOLOGY CONFIGURATION (/etc/slurm/topology.conf):
# Defines the network hierarchy — maps nodes to switches

SwitchName=spine01 Switches=tor[01-16]
SwitchName=spine02 Switches=tor[17-32]

SwitchName=tor01 Nodes=gpu[001-016]     # 16 nodes per ToR switch
SwitchName=tor02 Nodes=gpu[017-032]
# ... continues for all nodes ...

# COMMUNICATION COST BY NETWORK LEVEL:
#   Intra-node (NVLink):      ~0.05ms AllReduce   → TP groups MUST be here
#   Intra-ToR (1 IB hop):     ~0.15ms AllReduce   → PP stages ideal here
#   Cross-ToR (2 IB hops):    ~0.35ms AllReduce   → DP groups OK here
#   Cross-Spine (3 IB hops):  ~0.80ms AllReduce   → Avoid for TP/PP

# JOB SCRIPT — REQUEST TOPOLOGY-AWARE PLACEMENT:
#SBATCH --switches=8@00:30:00
# Meaning: use ≤8 ToR switches; wait up to 30 min for tight placement.
# After 30 min: use best available (don't delay forever for topology).

# For 64-node job (8 nodes/ToR): --switches=8 guarantees max 2-hop distance.
# For 128-node job:               --switches=16 allows 3-hop at worst.

# VERIFY TOPOLOGY PLACEMENT AFTER JOB STARTS:
scontrol show job $SLURM_JOB_ID | grep NodeList
# Then: ibnetdiscover | grep <node_name> to see which switch each node connects to

# MEASURING TOPOLOGY QUALITY:
# Run nccl-tests on the allocated nodes after job starts:
srun --ntasks=64 --ntasks-per-node=1 \
    /path/to/nccl-tests/build/all_reduce_perf -b 1G -e 1G -f 2 -g 8
# Alert: if busbw < 35 GB/s → placement spans too many switch hops
```

16.4.2 Gang Scheduling — All-or-Nothing Launch

```bash
```

```
# GANG SCHEDULING: All training ranks start simultaneously or none start.
# This prevents deadlocks where some ranks hold GPUs while others queue.

# SLURM's srun handles this natively:
# All tasks receive SIGCONT simultaneously after all are allocated.
# --kill-on-bad-exit=1 ensures cleanup if any rank fails at startup.

# VERIFY SIMULTANEOUS START (check timestamps):
sacct -j $SLURM_JOB_ID --format=JobID,Start,Elapsed --parsable2
# All subjobs should show Start within ±1 second.

# FOR STRICT GANG SCHEDULING (Kubernetes path — see §16.6):
# Use Volcano scheduler with minAvailable constraint.

# CHECKPOINT-RESTART FOR PREEMPTED JOBS:
# When production job preempts this job:
#   SLURM sends SIGTERM → job script saves checkpoint → job requeues
#   Job restarts from checkpoint (no training lost)

# Required training loop additions (Chapter 10 §10.10):
# signal.signal(signal.SIGTERM, save_checkpoint_and_exit)
# signal.signal(signal.SIGUSR1, save_checkpoint_and_exit)  # Pre-walltime warning
```

16.5 Kubernetes for AI — Architecture and GPU Resource Model

16.5.1 The Kubernetes GPU Stack

```
KUBERNETES GPU RESOURCE MODEL:

  USER WORKLOAD (Pod requesting GPUs)
           ↓
  Kubernetes API Server (pod spec: limits.nvidia.com/gpu: "8")
           ↓
  kube-scheduler (finds nodes with available GPUs)
           ↓
  NVIDIA GPU Device Plugin (DaemonSet on every node)
    • Advertises nvidia.com/gpu as schedulable resource
    • Maps GPU resource requests to physical device files
    • Handles /dev/nvidia[0-7] assignment to pods
           ↓
  containerd + NVIDIA Container Toolkit
```

```
    • Injects GPU libraries into container environment
    • Sets CUDA_VISIBLE_DEVICES inside container
    • Enables GPU access from container processes
          ↓
  Physical GPU Hardware (H100 SXM5)

SETUP COMMANDS:
  # Install NVIDIA GPU device plugin (mandatory on every GPU node):
  kubectl apply -f https://raw.githubusercontent.com/NVIDIA/k8s-device-plugin/\
    v0.15.0/nvidia-device-plugin.yml

  # Verify GPU resources visible to Kubernetes:
  kubectl describe node gpu-node-01 | grep -A 10 "Capacity:"
  # Expected:
  # Capacity:
  #   nvidia.com/gpu:  8
  # Allocatable:
  #   nvidia.com/gpu:  8

  # DCGM exporter for GPU monitoring (Prometheus metrics):
  helm install dcgm-exporter nvidia/dcgm-exporter \
    --set serviceMonitor.enabled=true
```

16.5.2 MIG — GPU Partitioning for Inference Multi-Tenancy

```bash
```

```
# MIG (Multi-Instance GPU) on H100:
# Partition one H100 into up to 7 independent isolated GPU instances.
# Each instance has dedicated HBM bandwidth and SM allocation.
# Isolation: one instance cannot affect another (errors, memory, compute).

# H100 MIG PROFILES:
# 1g.10gb:  1 GPU slice,  ~125 TFLOPS,  10 GB HBM  (7 instances max)
# 2g.20gb:  2 GPU slices, ~250 TFLOPS,  20 GB HBM  (3 instances max)
# 3g.40gb:  3 GPU slices, ~370 TFLOPS,  40 GB HBM  (2 instances max)
# 7g.80gb:  Full GPU                               (1 instance = full)

# ENABLE MIG MODE:
sudo nvidia-smi -i 0 --mig-mode=Enable
sudo reboot  # Required for MIG mode activation

# CREATE MIG INSTANCES (all 7 small instances):
sudo nvidia-smi mig -cgi 1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb
sudo nvidia-smi mig -cci  # Create compute instances for each GPU instance

# KUBERNETES MIG RESOURCE NAMES:
# nvidia.com/mig-1g.10gb   → small inference (7B model each)
# nvidia.com/mig-2g.20gb   → medium inference (13B model each)
# nvidia.com/mig-7g.80gb   → full GPU (70B model with TP=1)

# Pod requesting a MIG slice:
# resources:
#   limits:
#     nvidia.com/mig-1g.10gb: "1"  # Request one 1g.10gb MIG instance

# WHEN MIG IS APPROPRIATE:
#   Inference serving with multiple small models (7B-13B)
#   Development environments (multiple users on one GPU)
#   Cost optimization: 7 × 7B inference on one H100 vs 7 separate GPUs
#   NOT for training (MIG instances can't communicate via NVLink for GEMM)
```

16.6 Kubernetes Production Patterns

16.6.1 GPU Inference Deployment

yaml

```yaml
# vllm-inference-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llama3-70b-inference
  namespace: ml-serving
  labels:
    app: llama3-inference
    tier: production
spec:
  replicas: 4                              # 4 serving replicas
  selector:
    matchLabels:
      app: llama3-inference
  template:
    metadata:
      labels:
        app: llama3-inference
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"
        prometheus.io/path: "/metrics"
    spec:
      # Priority: production serving beats research training
      priorityClassName: gpu-production-critical

      containers:
        - name: vllm-server
          image: vllm/vllm-openai:v0.6.0
          command:
            - python
            - -m
            - vllm.entrypoints.openai.api_server
            - --model=/models/llama3-70b
            - --tensor-parallel-size=2        # TP=2: need 2 GPUs per replica
            - --quantization=fp8
            - --max-model-len=32768
            - --gpu-memory-utilization=0.90
            - --enable-prefix-caching
            - --enable-chunked-prefill
            - --port=8000

          resources:
```

```yaml
        requests:
          nvidia.com/gpu: "2"            # TP=2 → 2 GPUs per pod
          memory: "400Gi"
          cpu: "28"
        limits:
          nvidia.com/gpu: "2"
          memory: "400Gi"
          cpu: "28"

      ports:
        - containerPort: 8000
          name: http

      livenessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 120         # Wait for model load
        periodSeconds: 30
        failureThreshold: 3

      readinessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 90
        periodSeconds: 10
        failureThreshold: 6

      env:
        - name: NCCL_IB_DISABLE
          value: "0"
        - name: NCCL_NET_GDR_LEVEL
          value: "2"

      volumeMounts:
        - name: dshm
          mountPath: /dev/shm        # Required for NCCL shared memory
        - name: model-cache
          mountPath: /models

    # GPU node affinity
    nodeSelector:
      nvidia.com/gpu.product: "NVIDIA-H100-SXM5-80GB"
```

```yaml
      tolerations:
        - key: "nvidia.com/gpu"
          operator: "Exists"
          effect: "NoSchedule"

      # Spread replicas across nodes for HA
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app: llama3-inference
                topologyKey: kubernetes.io/hostname

      volumes:
        - name: dshm
          emptyDir:
            medium: Memory
            sizeLimit: "50Gi"
        - name: model-cache
          persistentVolumeClaim:
            claimName: llama3-model-cache-pvc

---
# HPA: Auto-scale based on GPU utilization via DCGM metrics
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: llama3-inference-hpa
  namespace: ml-serving
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: llama3-70b-inference
  minReplicas: 2
  maxReplicas: 16
  metrics:
    - type: External
      external:
        metric:
```

```yaml
          name: DCGM_FI_DEV_GPU_UTIL
          selector:
            matchLabels:
              app: llama3-inference
        target:
          type: AverageValue
          averageValue: "80"                # Scale out when avg GPU util > 80%
```

16.6.2 Multi-Node Distributed Training on Kubernetes

yaml

```yaml
# pytorch-training-job.yaml (using PyTorch Training Operator)
apiVersion: "kubeflow.org/v1"
kind: "PyTorchJob"
metadata:
  name: llama3-70b-pretrain
  namespace: ml-training
spec:
  pytorchReplicaSpecs:
    Master:
      replicas: 1
      restartPolicy: OnFailure
      template:
        metadata:
          annotations:
            volcano.sh/queue-name: "training-high-priority"
        spec:
          containers:
            - name: pytorch
              image: nvcr.io/nvidia/pytorch:24.03-py3
              command:
                - "torchrun"
                - "--nproc_per_node=8"
                - "--nnodes=$(WORLD_SIZE)"
                - "--node_rank=$(RANK)"
                - "--master_addr=$(MASTER_ADDR)"
                - "--master_port=29500"
                - "train.py"
              resources:
                limits:
                  nvidia.com/gpu: "8"
                  memory: "1800Gi"
                  cpu: "112"
              env:
                - name: NCCL_IB_DISABLE
                  value: "0"
                - name: NCCL_NET_GDR_LEVEL
                  value: "2"
              volumeMounts:
                - name: dshm
                  mountPath: /dev/shm
                - name: training-data
                  mountPath: /data
                - name: checkpoints
```

```yaml
            mountPath: /checkpoints
        volumes:
          - name: dshm
            emptyDir:
              medium: Memory
              sizeLimit: 100Gi         # Critical: NCCL shared memory buffer
          - name: training-data
            persistentVolumeClaim:
              claimName: training-dataset-pvc
          - name: checkpoints
            persistentVolumeClaim:
              claimName: checkpoint-storage-pvc
        nodeSelector:
          nvidia.com/gpu.product: "NVIDIA-H100-SXM5-80GB"
        tolerations:
          - key: "nvidia.com/gpu"
            operator: "Exists"
            effect: "NoSchedule"

    Worker:
      replicas: 63                          # 63 workers + 1 master = 64 nodes total
      restartPolicy: OnFailure
      template:
        # ... identical spec to Master ...
```

16.6.3 Gang Scheduling with Volcano

yaml

```yaml
# Volcano gang scheduling: ALL pods start simultaneously or NONE start.
# Prevents the deadlock where some ranks hold GPUs while others wait in queue.

# volcano-job.yaml
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: llama3-gang-training
  namespace: ml-training
spec:
  minAvailable: 64                        # ALL 64 pods must be schedulable at once
  schedulerName: volcano
  queue: training-high-priority

  policies:
    - event: PodFailed
      action: RestartJob                  # Restart ALL pods if ANY pod fails
    - event: PodEvicted
      action: RestartJob                  # Same for evictions (preemption)

  tasks:
    - name: worker
      replicas: 64
      minAvailable: 64
      template:
        spec:
          containers:
            - name: trainer
              image: nvcr.io/nvidia/pytorch:24.03-py3
              resources:
                limits:
                  nvidia.com/gpu: "8"
              command: ["bash", "/scripts/launch_training.sh"]
          nodeSelector:
            accelerator: nvidia-h100

# WITHOUT gang scheduling (default kube-scheduler):
#   Pods 1-50: scheduled and start, allocate 400 GPUs, wait for pod 51-64
#   Pods 51-64: can't schedule (not enough GPUs free)
#   DEADLOCK: allocated GPUs idle, new pods can't start
#   SOLUTION: minAvailable=64 → Volcano holds all 64 or none
```

16.7 Resource Isolation and Multi-Tenancy

16.7.1 Kubernetes Namespace Quotas

```yaml
```

```yaml
# namespace-quota.yaml — Per-team GPU resource limits
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-nlp-quota
  namespace: team-nlp
spec:
  hard:
    requests.nvidia.com/gpu: "64"        # Max 64 GPUs this namespace can request
    limits.nvidia.com/gpu: "64"
    requests.cpu: "1792"                  # 64 GPUs × 28 CPU cores
    requests.memory: "25600Gi"
    pods: "200"
    count/pytorchjobs.kubeflow.org: "10"

---
# LimitRange: enforce defaults and per-container maximums
apiVersion: v1
kind: LimitRange
metadata:
  name: gpu-limit-range
  namespace: team-nlp
spec:
  limits:
    - type: Container
      default:
        nvidia.com/gpu: "1"               # Default: 1 GPU per container
        memory: "64Gi"
        cpu: "14"
      defaultRequest:
        nvidia.com/gpu: "1"
      max:
        nvidia.com/gpu: "8"               # No single container can request >8 GPUs

---
# PriorityClass hierarchy
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: gpu-production-critical
value: 1000000
preemptionPolicy: PreemptLowerPriority
description: "Production inference — preempts research training"
```

```yaml
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: gpu-research-training
value: 100000
preemptionPolicy: PreemptLowerPriority
description: "Research training — can be preempted by production"
```

16.7.2 SLURM Multi-Tenant Design

```bash
# ACCOUNT HIERARCHY FOR 5-TEAM CLUSTER:
#
#   root (1000 GPU quota total)
#   ├── team_training  (500 GPU soft limit)
#   |   ├── team_nlp   (200 GPUs)
#   |   └── team_cv    (200 GPUs)
#   └── team_inference (300 GPU guaranteed)
#       ├── team_prod  (200 GPUs reserved)
#       └── team_dev   (100 GPUs)

sacctmgr add cluster name=gpu_cluster
sacctmgr add account name=root Cluster=gpu_cluster
sacctmgr add account name=team_training Parent=root
sacctmgr add account name=team_inference Parent=root
sacctmgr add account name=team_nlp Parent=team_training GrpTRES="gres/gpu=200"
sacctmgr add account name=team_prod Parent=team_inference GrpTRES="gres/gpu=200"

# ISOLATION RULES:
# GrpTRES: hard limit — team cannot exceed this at any time
# MaxTRESPU: per-user limit within their team quota
# QOS preempt: production can reclaim GPUs from training

# SLURM RESERVATION for guaranteed capacity:
# Reserve 100 GPUs permanently for production inference:
scontrol create reservation StartTime=2026-01-01T00:00:00 \
    Duration=UNLIMITED Nodes=gpu[100-112] \
    Accounts=team_prod Flags=IGNORE_JOBS,ANY_NODES
```

16.8 Fault-Tolerant Training — Checkpoint-Restart Automation

16.8.1 Signal-Driven Checkpoint Pattern

```python
import signal
import sys
import torch.distributed as dist

CHECKPOINT_DIR = "/lustre/checkpoints"

def save_checkpoint_and_exit(signum, frame):
    """
    Called on SIGTERM (SLURM preemption) or SIGUSR1 (pre-walltime warning).
    Saves checkpoint and exits cleanly so SLURM can requeue the job.
    """
    rank = dist.get_rank() if dist.is_initialized() else 0
    if rank == 0:
        print(f"Signal {signum} received — saving checkpoint at step {step}")
        torch.save({
            "step": step,
            "model": model.state_dict(),
            "optimizer": optimizer.state_dict(),
            "scheduler": scheduler.state_dict(),
            "rng_state": torch.cuda.get_rng_state(),
        }, f"{CHECKPOINT_DIR}/emergency_step{step}.pt")
        print("Checkpoint saved. Exiting cleanly.")

    if dist.is_initialized():
        dist.barrier()  # All ranks synchronize before exit

    sys.exit(0)  # Clean exit → SLURM sees success → requeues with #SBATCH --requeue

# Register signal handlers (before training loop):
signal.signal(signal.SIGTERM, save_checkpoint_and_exit)  # Preemption
signal.signal(signal.SIGUSR1, save_checkpoint_and_exit)  # Pre-walltime warning

# #SBATCH --signal=USR1@120 in job script sends SIGUSR1 120s before walltime.
# Training saves checkpoint → exits → SLURM requeues from checkpoint.
# Effective: 2-minute grace before walltime; near-zero lost compute.
```

16.8.2 Automatic Checkpoint Detection and Resume

```python
import os
from pathlib import Path

def find_latest_checkpoint(checkpoint_dir: str) -> str | None:
    """Find the most recent valid checkpoint in the checkpoint directory."""
    ckpt_dir = Path(checkpoint_dir)
    if not ckpt_dir.exists():
        return None

    checkpoints = sorted(ckpt_dir.glob("*.pt"), key=lambda p: p.stat().st_mtime)
    if not checkpoints:
        return None

    # Validate the latest checkpoint (Chapter 10 §10.10 protocol)
    for ckpt in reversed(checkpoints):  # Try newest first
        try:
            state = torch.load(ckpt, map_location="cpu")
            assert "model" in state and "optimizer" in state
            return str(ckpt)
        except Exception as e:
            print(f"Checkpoint {ckpt} invalid: {e}. Trying next.")

    return None  # All checkpoints invalid → start from scratch

# At job startup:
checkpoint_path = find_latest_checkpoint(CHECKPOINT_DIR)
if checkpoint_path:
    print(f"Resuming from: {checkpoint_path}")
    state = torch.load(checkpoint_path, map_location="cpu")
    model.load_state_dict(state["model"])
    optimizer.load_state_dict(state["optimizer"])
    start_step = state["step"]
else:
    print("No checkpoint found — starting from scratch")
    start_step = 0
```

16.9 SLURM vs Kubernetes — Decision Framework

```
COMPREHENSIVE COMPARISON:
```

Dimension	SLURM	Kubernetes
Primary use case	HPC, large-scale training	Cloud-native, inference
Job model	Batch jobs, MPI	Microservices + batch (Operators)
GPU scheduling	Native GRES	NVIDIA device plugin
Gang scheduling	Built-in	Requires Volcano/Kueue
Multi-node training	Excellent (native srun)	Good (PyTorch Operator)
Topology awareness	Native (topology.conf)	Limited (improving)
Preemption	Mature (QOS-based)	PriorityClasses
Resource accounting	sacctmgr, fair-share	ResourceQuota + RBAC
Container support	Singularity/Apptainer	Docker/containerd native
Auto-scaling	Manual node add	Excellent (Cluster Autoscaler)
Service discovery	N/A	Excellent (DNS, Services)
Monitoring	sacct + external	Prometheus/Grafana native
Multi-tenancy	Accounts + QOS	Namespaces + RBAC + Quotas
Industry adoption	Research labs, HPC, NVIDIA	Cloud providers, inference
Production examples	Meta (training), NERSC	Google, OpenAI (serving), AWS

```
USE SLURM WHEN:
  ✓ Large-scale training jobs (> 100 GPUs) — native gang scheduling
  ✓ HPC cluster with InfiniBand and topology-aware placement
  ✓ Job accounting and fair-share scheduling are critical
  ✓ Team uses MPI or torchrun workflows on bare metal
  ✓ Research environment with diverse, heterogeneous workload types

USE KUBERNETES WHEN:
  ✓ Inference serving — microservice model, auto-scaling HPA
  ✓ Multi-tenant platform with strong namespace isolation
  ✓ CI/CD pipelines for model evaluation and testing
  ✓ Cloud deployment (EKS, GKE, AKS are all Kubernetes native)
  ✓ Mixed CPU + GPU workloads in one orchestration system
  ✓ Team has existing Kubernetes expertise and tooling

USE BOTH (hybrid, common at scale):
  Training cluster:  SLURM (bare-metal, native gang scheduling)
  Inference cluster: Kubernetes (cloud-native, auto-scaling)
  Bridge:            MLFlow / Kubeflow for experiment tracking across both

  Meta's architecture:
```

```
    FBLearner Flow: custom orchestrator providing SLURM-like semantics
    Service layer: Kubernetes-style container management
    Result: best of both worlds, higher engineering investment
```

16.10 Production Operations Playbook

16.10.1 Daily Cluster Health Checks

```bash
```

```bash
#!/bin/bash
# cluster_health_check.sh — Run daily via cron

echo "=== GPU CLUSTER HEALTH $(date) ==="

# ── SLURM NODE STATE ──────────────────────────────────────────────
echo "NODE STATES:"
sinfo --format="%P %n %T %C %G" | column -t

echo "DRAINED NODES (require investigation):"
sinfo --states=drained,down --format="%n %T %E" | column -t

# ── QUEUE SUMMARY ─────────────────────────────────────────────────
PENDING=$(squeue -h -t PENDING | wc -l)
RUNNING=$(squeue -h -t RUNNING | wc -l)
echo "Queue: $RUNNING running, $PENDING pending"

# Alert: jobs pending > 1 hour (potential scheduling stall)
squeue -h -t PENDING --format="%i %u %P %j %r" | while read line; do
    jobid=$(echo $line | awk '{print $1}')
    submit_time=$(scontrol show job $jobid | grep -o "SubmitTime=[^ ]*" | cut -d= -f
    if [ -n "$submit_time" ]; then
        age_seconds=$(( $(date +%s) - $(date -d "$submit_time" +%s) ))
        if [ $age_seconds -gt 3600 ]; then
            echo "ALERT: Job $jobid pending > 1 hour: $line"
        fi
    fi
done

# ── GPU UTILIZATION VIA DCGM ──────────────────────────────────────
echo "CLUSTER GPU UTILIZATION:"
dcgmi dmon -e 1003,1005 -c 1 | \
    awk '{sm+=$2; bw+=$3; count++} END \
    {printf "Avg SM util: %.1f%%, Avg BW util: %.1f%%\n", sm/count, bw/count}'

# ── KUBERNETES HEALTH ─────────────────────────────────────────────
if command -v kubectl &>/dev/null; then
    echo ""
    echo "K8S POD SUMMARY (ml-serving namespace):"
    kubectl get pods -n ml-serving --no-headers | \
        awk '{state[$3]++} END {for(s in state) printf "%s: %d\n", s, state[s]}'

    # Alert: pods stuck in Pending > 10 minutes
```

```
    kubectl get pods -n ml-serving -o json | jq -r '
        .items[] |
        select(.status.phase == "Pending") |
        select(.spec.containers[].resources.limits["nvidia.com/gpu"] != null) |
        "\(.metadata.name): pending since \(.metadata.creationTimestamp)"'
fi
```

16.10.2 Essential Command Reference

```
SLURM PRODUCTION COMMANDS:

Job Management:
  sbatch script.sh              Submit batch job
  squeue -u $USER               View my jobs
  squeue -p gpu_train           View jobs in partition
  scancel <JOBID>               Cancel a job
  scontrol show job <JOBID>     Detailed job info (nodes, time, reason)
  sacct -j <JOBID>              Job accounting after completion
  scontrol requeue <JOBID>      Manually requeue a job

Cluster Admin:
  sinfo                         Node/partition overview
  sinfo -N -l                   Per-node details
  sshare -a -l                  Fair-share standings
  scontrol update NodeName=gpu001 State=DRAIN Reason="ECC errors"
  scontrol update NodeName=gpu001 State=RESUME

KUBERNETES PRODUCTION COMMANDS:

Pod Management:
  kubectl get pods -n ml-training --watch        Monitor pod states live
  kubectl describe pod <POD> -n ml-training       Debug: events, scheduling
  kubectl logs <POD> -n ml-training -f            Follow pod logs
  kubectl exec -it <POD> -- nvidia-smi            Check GPU state in pod
  kubectl exec -it <POD> -- bash                  Interactive shell

Resource Info:
  kubectl describe node <NODE>                    GPU capacity/allocations
  kubectl get resourcequota -n team-nlp           Quota usage
  kubectl top nodes                               CPU/mem (not GPU — use DCGM)
  kubectl get priorityclass                       Priority tier definitions

Debugging:
```

```
  kubectl get events -n ml-training --sort-by=.lastTimestamp
  kubectl describe pytorchjob <JOB> -n ml-training
  kubectl get pods -o wide -n ml-training         Node placement
```

16.11 Key Takeaways and Review Questions

```
KEY PRINCIPLES:

  1. SCHEDULER EFFICIENCY IS WORTH MORE THAN GPU GENERATION:
     60% vs 85% cluster utilization at constant cost.
     25% more effective compute from scheduling alone.
     Fair-share + QOS + topology-aware placement are the three levers.

  2. GANG SCHEDULING IS MANDATORY FOR DISTRIBUTED TRAINING:
     Without it: partial rank allocation → deadlock → GPUs idle for hours.
     SLURM: built-in via srun synchronization.
     Kubernetes: requires Volcano with minAvailable constraint.

  3. TOPOLOGY-AWARE PLACEMENT AFFECTS MFU DIRECTLY:
     TP groups within NVLink: 0.05ms AllReduce.
     TP groups across switches: 0.35-0.80ms AllReduce.
     At 160 AllReduces/step: 3 switches vs 1 switch = +120ms per step.
     SLURM: --switches=N is the most important job parameter after GPU count.

  4. PREEMPTION REQUIRES CHECKPOINT-RESTART AUTOMATION:
     Production jobs must preempt research jobs (QOS/PriorityClass).
     Research jobs must checkpoint cleanly on SIGTERM.
     Without this: preemption wastes the preempted job's progress.
     SLURM: --signal=USR1@120 + --requeue.
     Kubernetes: PriorityClass + gracefulTerminationPeriodSeconds.

  5. SLURM FOR TRAINING, KUBERNETES FOR INFERENCE (the standard split):
     SLURM: native gang scheduling, InfiniBand topology, fair-share accounting.
     Kubernetes: auto-scaling, service mesh, cloud-native operator ecosystem.
     Hybrid with experiment tracking bridge: best of both worlds.

  6. MULTI-TENANCY REQUIRES BOTH HARD LIMITS AND FAIR PRIORITY:
     Hard limits (ResourceQuota / GrpTRES): prevent resource monopoly.
     Fair-share (SLURM) / PriorityClasses (K8s): ensure equitable access.
     Isolation (MIG / Namespaces): prevent cross-tenant interference.
```

Review Questions

Difficulty: Intermediate

Question 1 (SLURM Debugging): A 512-GPU distributed training job on SLURM fails with NCCL timeout errors after approximately 2 hours. The job uses 64 nodes. Logs show `NCCL WARN Timeout in ncclAllReduce`. Provide three specific, actionable configuration changes.

Expected answer: (1) SLURM job script: add `#SBATCH --switches=8@00:30:00` — constrains the job to 8 ToR switches (ensuring max 2-hop IB path). Without this, nodes may span 3-4 switch hops → AllReduce latency increases → NCCL timeout triggers. Also add `#SBATCH --requeue` so the job restarts if a node fails mid-training. (2) NCCL environment: set `NCCL_TIMEOUT=1800` (30 minutes instead of default 300 seconds). At 64 nodes, checkpoint writes and straggler nodes can legitimately cause >5-minute gaps between collectives. The default 300s timeout is too aggressive for large clusters. Also verify `NCCL_IB_HCA` is correctly pinned to the rail-aligned NIC order. (3) Node health check: run `ibstatus` on all 64 nodes — check for any link operating at < 400 Gbps. A single degraded IB link causes one rank to be slower than others, which causes all ranks to wait at the AllReduce barrier → accumulating delays → timeout after 2 hours. Add the prolog script that validates GPU and NIC health before job starts.

Difficulty: Principal Level

Question 2 (Multi-Tenant SLURM Design): Design the SLURM account hierarchy and QOS configuration for a 1,024-GPU cluster shared by 5 teams: (A) LLM Training (500 GPU target, preemptible), (B) Production Inference (200 GPU guaranteed SLA), (C) Research (150 GPU), (D) Fine-tuning (100 GPU), (E) Development (74 GPU, short jobs only). Write the key sacctmgr and QOS commands. Explain which teams can preempt which, and how fair-share prevents monopoly.

Expected answer: Account hierarchy: root → (training_pool: 650 GPUs, inference_pool: 350 GPUs). training_pool → team_llm (500), team_research (150), team_finetune (100). inference_pool → team_prod (200), team_dev (74). QOS design: team_prod: Priority=200, no preemption, GrpTRES=gres/gpu=200. This is the guaranteed tier — never preempted. team_llm: Priority=50, Preempt=research (can preempt research but not prod), MaxWall=30 days. team_research: Priority=30, PreemptMode=requeue (gets preempted by llm and prod). team_finetune: Priority=40, MaxWall=7 days. team_dev: MaxTRESPU=gres/gpu=8, MaxWall=4h (short-job enforcement).

```
sacctmgr add qos name=prod_inference Priority=200 GrpTRES="gres/gpu=200"
sacctmgr add qos name=llm_training Priority=50 \
    MaxTRESPU="gres/gpu=500" Preempt=research PreemptMode=requeue
sacctmgr add qos name=research Priority=30 MaxTRESPU="gres/gpu=150"
sacctmgr add qos name=dev Priority=20 MaxTRESPU="gres/gpu=8" \
    MaxWallDurationPerJob=4:00:00
```

Fair-share: PriorityWeightFairshare=10000. Team LLM runs a 30-day 500-GPU job → high RawUsage → FairShare drops toward 0 → lower priority → Research and Finetune get higher priority the following week. This prevents team LLM from monopolizing even within their quota. At the end of the week: LLM's usage decays (half-life 7 days), FairShare recovers.

Difficulty: Principal Level

Question 3 (Kubernetes vs SLURM Trade-off): Your organization has a SLURM training cluster (512 H100s) and needs to build an inference serving platform for 20 production LLM services. An engineer proposes using the existing SLURM cluster for both training and inference. Evaluate this proposal and design the correct architecture.

Expected answer: The SLURM-for-inference proposal is technically feasible but wrong for production serving. Core problems: (1) SLURM batch jobs have no native auto-scaling based on request load. Inference traffic is bursty; you need HPA to scale from 2 to 20 replicas during peak hours. SLURM would require manually submitting new batch jobs or custom wrappers — no Kubernetes-native HPA. (2) SLURM provides no service mesh, no DNS-based service discovery, no load balancing. Each LLM service needs an endpoint for routing; SLURM provides none of these. (3) Kubernetes deployments support rolling updates (update model weights without downtime); SLURM jobs must be cancelled and resubmitted. (4) Production SLAs (TTFT P99 < 300ms) require admission control and priority queuing integrated with the serving stack — Kubernetes probes and readiness gates do this naturally. Correct architecture: keep SLURM for training. Deploy a separate Kubernetes cluster for inference. Scale: inference doesn't need 512 H100s simultaneously; start with 32-64 H100s in the K8s cluster, auto-scale with HPA. Bridge via a shared Lustre filesystem: training outputs checkpoints to /lustre/checkpoints/, inference K8s pods mount the same filesystem to load model weights. MLflow or Weights & Biases track experiments across both clusters. This hybrid architecture is exactly what Meta (FBLearner + service mesh) and most large AI organizations deploy.

Difficulty: Principal Level

Question 4 (Gang Scheduling Failure Mode): A Kubernetes cluster has 256 GPUs available. A 64-GPU PyTorch training job submits 64 pods. The default kube-scheduler begins placing pods.

After scheduling 50 pods, it cannot find nodes for the remaining 14 (another team's inference pods hold 14 GPUs). Describe in detail: (a) what happens to training throughput, (b) why this is worse than a clean failure, (c) how Volcano with minAvailable=64 resolves it.

Expected answer: (a) The 50 pods that were scheduled start their torchrun processes. They attempt NCCL init by connecting to the master rank. Ranks 0-49 are present; ranks 50-63 are missing. NCCL waits for all 64 ranks to connect within NCCL_TIMEOUT seconds. During this wait: all 50 running pods consume 400 GPUs at full power (CUDA context initialized, memory allocated, Tensor Cores idle) while computing nothing. This is 400 GPU-hours of pure waste for every NCCL_TIMEOUT duration. (b) This is worse than a clean failure for two reasons: (1) The 50 allocated pods hold 400 GPUs that cannot be used by other jobs. The inference team's 14 GPUs are also allocated. Total locked up: 414 GPUs, none doing useful work. A clean failure would have released all resources immediately. (2) This can create cascading deadlocks: if the inference team's 14 GPU jobs also need additional GPUs for their next operation, now neither training nor inference can make progress. The entire cluster deadlocks. (c) Volcano with `spec.minAvailable: 64` implements gang scheduling: Volcano holds all 64 pods in a "pending gang" state. It does NOT begin scheduling any pods until all 64 can be scheduled simultaneously. While waiting: no resources are consumed (pods not yet bound to nodes). If 64 GPUs are not available now: Volcano waits until they become available (inference job completes). Then: all 64 pods start simultaneously within seconds. Zero GPU waste from partial allocation. This is the correct solution — it exchanges scheduler complexity for guaranteed no-deadlock gang semantics.

Further Reading

1. *SLURM Workload Manager Documentation* — SchedMD. `slurm.schedmd.com/documentation.html` Read: `slurm.conf` (especially Prolog/Epilog, GRES, topology), `priority_multifactor.html`, and `topology.html`.
2. *NVIDIA GPU Operator for Kubernetes* — NVIDIA (2024). `docs.nvidia.com/datacenter/cloud-native/gpu-operator/` Automates device plugin, driver, DCGM exporter deployment on Kubernetes clusters.
3. *Volcano: A Kubernetes-Native Batch System* — CNCF project. `volcano.sh/docs` Gang scheduling, queues, job lifecycle. Read the JobPlugin and Queue documentation.
4. *PyTorch Training Operator (Kubeflow)* — Kubeflow (2024). `www.kubeflow.org/docs/components/training/` The PyTorchJob spec and elastic training support.
5. *Scheduling Best Practices for Large-Scale ML on Kubernetes* — Google Cloud Blog (2023). Practical topology-aware scheduling patterns for GKE GPU clusters that apply equally to on-premise Kubernetes.

— End of Chapter 16 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved